

SINR: Sparsity Driven Compressed Implicit Neural Representations

Dhananjaya Jayasundara, Sudarshan Rajagopalan, Yasiru Ranasinghe,
 Trac D. Tran, and Vishal M. Patel
 Johns Hopkins University

{vjayasul, sambasa2, dranasil, trac, vpatel36}@jhu.edu

Abstract

*Implicit Neural Representations (INRs) are increasingly recognized as a versatile data modality for representing discretized signals, offering benefits such as infinite query resolution and reduced storage requirements. Existing signal compression approaches for INRs typically employ one of two strategies: 1. direct quantization with entropy coding of the trained INR; 2. deriving a latent code on top of the INR through a learnable transformation. Thus, their performance is heavily dependent on the quantization and entropy coding schemes employed. In this paper, we introduce **SINR**, an innovative compression algorithm that leverages the patterns in the vector spaces formed by weights of INRs. We compress these vector spaces using a high-dimensional sparse code within a dictionary. Further analysis reveals that the atoms of the dictionary used to generate the sparse code do not need to be learned or transmitted to successfully recover the INR weights. We demonstrate that the proposed approach can be integrated with any existing INR-based signal compression technique. Our results indicate that **SINR** achieves substantial reductions in storage requirements for INRs across various configurations, outperforming conventional INR-based compression baselines. Furthermore, **SINR** maintains high-quality decoding across diverse data modalities, including images, occupancy fields, and Neural Radiance Fields.*

1. Introduction

Despite the fact that all naturally occurring signals observed by humans are continuous, capturing these signals through digital devices requires their discretization. For example, an image of a mountain is processed and stored in a discretized format. A primary reason for this approach is to conserve storage space; storing signals with high precision in an almost continuous manner would necessitate a sub-

stantial amount of storage. Consequently, the digital representation of signals is both practical and essential. For instance, it is estimated that over 400TB of data is created every day [8]. Moreover, humans share their captured signals through various mediums on a daily basis. Therefore, data compression is essential for efficient transmission.

Traditional signal compression techniques often rely on classic signal processing methods and are modality-specific. For example, JPEG [39], designed for photographic images, and is unsuitable for audio files. Similarly, audio compression standards like MP3 or AAC [4] are optimized for sound and are not applicable to images. With the advancements of neural networks, researchers have explored compressing signals using neural methods, predominantly through mechanisms based on autoencoders [2, 6, 7, 25, 37, 41]. In these systems, the encoder transforms the signal into a latent vector, which the decoder then uses to reconstruct the original signal. While autoencoder-based methods effectively encode signals into latent vectors, they are generally designed for a single modality. Adapting these methods to different data modalities not only requires training on a large corpus of data specific to those modalities but also a specialized autoencoder architecture tailored to handle the data effectively.

In recent years, there has been a significant surge in interest in representing signals through Implicit Neural Representations (INRs). Unlike large models based on autoencoders, INRs typically consist of multi-layer perceptrons (MLPs) equipped with specialized nonlinearities that differ from the conventional nonlinearities used in deep learning. This simplicity and versatility allow INRs to unify signal representations across diverse data modalities. When signals are represented by INRs, they are encoded in the MLPs' weights and biases. For instance, in image transmission, instead of using conventional JPEG encoding, the weights and biases of the MLP are transmitted by a transmitter (TX). A receiver (RX) can then feed the coordinates into the MLP and decode the image. The primary advantage of INRs lies in their ability to represent signals with high fidelity while utilizing fewer parameters than parameter-

heavy autoencoder-based mechanisms.

Recent advances in INR-based signal compression include COIN [9], COIN++ [10], INRIC [34], and SHACIRA [11]. COIN pioneered the application of INRs for image compression. Building on this, COIN++ and INRIC introduced quantization and entropy coding to improve compression efficiency. Both approaches also focus on enhancing the generalization capabilities of INRs through meta-learning techniques. Additionally, COIN++ incorporates latent modulations discovered via a learnable transformation applied on top of the INR model. However, COIN++ requires transmitting the base INR and the learned transformation apriori, in addition to the latent modulations for signal decoding. Alternatively, SHACIRA applies quantization on the latent weights and enforces entropy regularization to reparameterize feature grids to enable efficient compression across diverse domains. None of the existing methods; however, have explored fundamentally compressing the INR by identifying patterns within its parameter space before applying standard techniques such as quantization and entropy coding.

In our work, named **SINR**, we build upon the observed behaviors of the vector spaces generated by the weights in an INR. We integrate compressed sensing algorithms into the INR-based compression pipeline, proposing a mechanism that obtains a higher-dimensional sparse code for the weight vectors without requiring any learnable transformations. Furthermore, based on the Central Limit Theorem (CLT) [42], the transformation matrix can be directly sampled from a distribution rather than learning or hand crafting it. This eliminates the need to transmit the transformation matrix for accurately decoding weight spaces. Consequently, **SINR**, as a fundamental compression technique built on the observations of weights spaces, achieves superior compression and higher decoding quality for each data modality compared to the competing methods. Moreover, it can be easily embedded into any INR-based signal compression algorithm.

2. Related works

2.1. Implicit neural representations

INRs have recently gained considerable attention in the computer vision community due to their streamlined network architectures and improved performance in various vision tasks compared to traditional, parameter-heavy models [13, 29, 32]. This surge in interest followed the advent of Neural Radiance Fields (NeRF) [24], which has inspired a plethora of subsequent studies [26, 45]. Further research has explored the pivotal role of different activation functions in INRs [28, 29, 32, 35]. Moreover, INRs provide a universal framework for representing various data modalities. More recent studies have investigated the use of INRs for image classification by transforming standard image for-

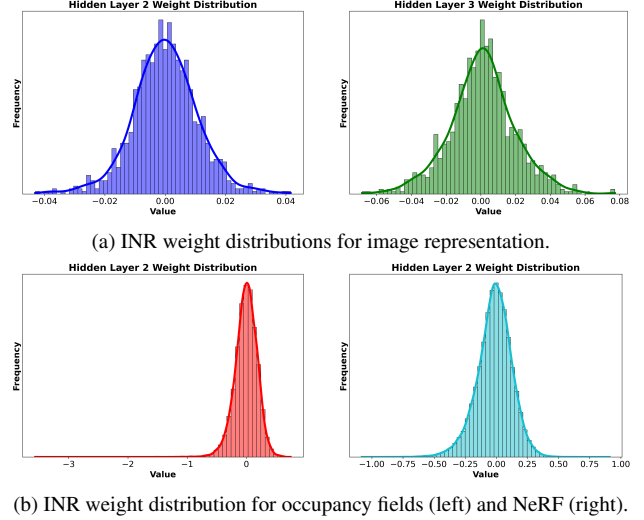


Figure 1. **Weight distribution of INRs** tends to follow a Gaussian distribution for various data modalities.

malts into INRs and training classifiers directly on the INRs' weights and biases [31]. These innovative approaches have showcased the potential of INRs to significantly reduce the dimensionality and computational complexity typically associated with conventional image processing techniques.

2.2. Signal compression

Signal compression is crucial for reducing bandwidth needs and saving storage space. With the rise of deep learning, signal compression has evolved into two main approaches: rule-based (traditional) and learning-based methods. Traditional compression methods, such as JPEG for images and MP3 for audio, rely on algorithmic techniques tailored to specific signal types. JPEG minimizes redundancies using the discrete cosine transform [27], while MP3 [4] employs a psycho-acoustic model that enhances compression by removing inaudible sounds through auditory masking. On the other hand, deep learning-based techniques use models trained on vast datasets, adapting to a wide range of signals without predefined algorithms. These methods offer flexibility but require different architectures for each data modality, presenting unique challenges. In this landscape, INRs stand out as a potential universal signal representer.

2.3. Compressed sensing

Compressed sensing is a field that capitalizes on the inherent sparsity of data to capture information efficiently. In digital imaging, not every pixel is crucial for accurate image reconstruction. Although images appear dense in pixel space, they exhibit considerable redundancy when transformed into different basis functions. This sparsity is exploited by compressed sensing algorithms to reconstruct the original image from fewer sampled data points. These algo-

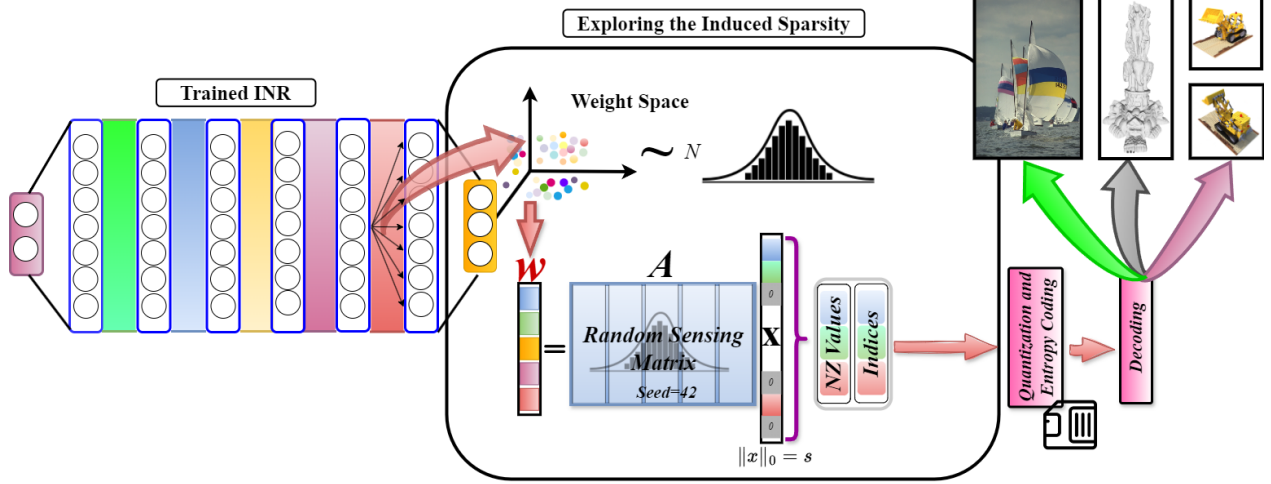


Figure 2. **The proposed SINR compression algorithm:** Standard compression techniques for INRs typically involve direct quantization and entropy coding of their weights. However, since natural signals exhibit inherent compressibility in a dictionary, the characteristics that aid in the compressibility of the weight space of an INR are discovered through the Gaussian nature of the weight space. Therefore, **SINR** employs L_1 minimization to identify a higher-dimensional sparse code. Furthermore, based on the weight space observations and the CLT, we simplify the encoding and decoding process using a random sensing matrix controlled by a seed. Subsequently, only the non-zero (NZ) values and their corresponding indices are quantized and entropy coded.

gorithms employ optimization techniques and linear algebra to solve underdetermined systems, revolutionizing data acquisition in areas such as medical imaging and signal processing. Dictionary learning, integral to compressed sensing, seeks sparse representations of data using dictionary elements or atoms that capture the data’s intrinsic structure. These atoms are either predefined or adaptively learned. Compressed sensing’s versatility is evident in its applications across various domains, such as image and video compression [44], medical image encryption [17], and classification tasks [14, 15, 19, 21]. It also addresses inverse vision problems like image inpainting [30], deblurring [16, 23], and super-resolution [3]. Recent efforts have merged dictionary learning with deep learning to tackle more complex computer vision challenges, including image recognition [36], denoising [43], and scene recognition [22]. These developments underscore compressed sensing’s transformative impact on computer vision.

Our work, **SINR**, is pioneering the application of compressed sensing principles to INRs. By leveraging these principles alongside the structural distributions of INR weights, **SINR** identifies redundancies in these spaces, resulting in substantial compression improvements.

3. Method

3.1. Signal representation through INRs

Mathematically, an INR can be defined by a function G_θ , where θ are the optimizable parameters of the neural net-

work. The input and output dimensions of G_θ vary for different data modalities. In general, G_θ acts as a mapping from an a -dimensional input coordinate space to a b -dimensional output signal space, described as:

$$G_\theta : \mathbb{R}^a \rightarrow \mathbb{R}^b.$$

For instance, for RGB images, $a = 2$ and $b = 3$, while for audio signals, $a = 1$ and $b = 1$. In this architecture, the output of the i^{th} layer, which feeds into the $(i + 1)^{\text{th}}$ layer, can be expressed as $\sigma(W^{(i)}y^{(i)} + b^{(i)})$. Here, σ denotes the activation function, and $y^{(i)}$ represents the output from the preceding layer. Furthermore, the choice of activation function (σ) plays a critical role in shaping the neural network’s ability to model complex functions, as explored in various studies [28, 29, 32, 35].

3.2. Exploring the compressibility

According to compressed sensing theory, most real-world signals display sparsity when transformed into an appropriate domain, meaning they can be accurately represented with fewer measurements than traditionally required. Furthermore, real-world signals can be compressed through a set of basis functions, and the coefficients of these functions are derived by minimizing the reconstruction loss. The core concept of INRs involves encoding signals into the weights and biases of an MLP. This process can be viewed as a classical domain transformation technique where pixel values are reconstructed by feeding the corresponding coordinates through the MLP. Unlike predefined signal trans-

formers like Fourier [5] and DCT [20], the MLP attempts to minimize the reconstruction loss through backpropagation to find the transformation. The learned representation of the signal resides in another domain. Given that real-world signals inherently exhibit sparsity in transformed domains, we hypothesize that this sparsity can be explored within the MLP's weights. If we can identify where this sparse nature is hidden within the weight space, we could achieve further compression on INRs compared to competing methods. However, identifying this sparse representation within the weights is not straightforward. We believe there are two main approaches to achieving a sparser representation, each with its own challenges and considerations.

The first approach involves either promoting or enforcing a specified level of sparsity in the weights during the training of an INR. Promoting sparsity can generally be achieved by incorporating L_1 regularization on the model parameters, which encourages many weights to approach zero, thereby creating a sparse representation. However, while we observed that L_1 regularization results in a higher level of sparsity within the weights, it fails to accurately represent the images. Alternatively, enforcing a specified level of sparsity can also be achieved through model pruning, where weights deemed insignificant are pruned or eliminated during the training process. When it comes to model pruning, we employed both structured and unstructured pruning of weights. We noted that both methods led to significant performance degradation for certain data modalities, particularly for occupancy fields. Moreover, only a small pruning percentages resulted in satisfactory performance for signal representation. For applications that require a high level of generalization, such as NeRFs, the pruning approach did not generalize well, indicating its limitations in achieving a balance between sparsity and performance.

The second approach seeks to uncover the inherent structures within the weights that aid in INR compression. This involves identifying patterns or regularities that can be exploited to reduce the dimensionality of the representation without sacrificing performance. We examined this from a dimensionality reduction perspective; however, the weight space in reduced dimensions did not reveal clear patterns, even across different natural images. Nonetheless, we observed that the weight space of an INR often tends to follow a normal distribution. Fig. 1 shows the weight distribution of the hidden layers of an INR when different data modalities are encoded into it. This suggests that INRs share a common pattern across different data modalities, showcasing a potential pathway for a fundamental compression. Given that each weight vector of an INR exhibits Gaussian behavior, we seek a higher-dimensional but sparse equivalent through a dictionary learning-based approach. Let us denote $\mathbf{w} \in \mathbb{R}^{k_1}$ as a hidden weight vector, $\mathbf{A} \in \mathbb{R}^{k_1 \times k_2}$ as

a dictionary, and $\mathbf{x} \in \mathbb{R}^{k_2}$ as the corresponding sparse vector. In search of a sparse representation, according to standard compressed sensing, we can write $\mathbf{w} = \mathbf{A}\mathbf{x}$, where $\|\mathbf{x}\|_0 < k_1$. To discover the sparse code $\mathbf{x}$, the best and most efficient choice is L_1 minimization, as L_0 minimization iterates through all possible combinations and is therefore not efficient. However, the problem arises with the sensing matrix, commonly referred to as the dictionary $\mathbf{A}$. Although we could use either a dictionary learning-based approach for learning basis functions for the dictionary or a deep learning-based learnable transformation, these approaches would be time-consuming. Furthermore, a TX needs to transmit the learned dictionary alongside the obtained sparse codes. Conversely, we propose that the dictionary does not need to be learned or even transmitted.

As we have confirmed, the weights are normally distributed. According to the CLT, a normally distributed random variable can be produced through a finite linear combination of any random variables. In summation form, this can be expressed as: $w_i = \sum_{j=1}^{k_2} A_{ij}x_j$, where w_i is the i -th element of the weight vector $\mathbf{w}$, A_{ij} is the element in the i -th row and j -th column of the sensing matrix $\mathbf{A}$, and x_j is the j -th element of the vector $\mathbf{x}$. To satisfy the CLT, the number of terms in the summation, which is k_2 , should be sufficiently large. Therefore, considering all elements of the weight vector $\mathbf{w}$, this can be compactly written as $\mathbf{w} = \mathbf{A}\mathbf{x}$. From CLT, the sensing matrix can be defined by a set of random vectors whose appropriate coefficients ($\mathbf{x}$) can be learned using sparse signal recovery algorithms such as matching pursuit or its variants. Therefore, the optimization can be written as, $\min \|\mathbf{x}\|_1$ subject to $\mathbf{w} = \mathbf{A}\mathbf{x}$.

For convenience, let us denote $\|\mathbf{x}\|_0$ as s . A further constraint to the above optimization procedure is that when the sparse code $\mathbf{x}$ is found, we need to store not only its non-zero elements but also the corresponding indices. Therefore, the above L_1 minimization is solved with $2s < k_1$. We do not apply our compression algorithm to the biases of the INR as the size of bias vectors is very small compared to those of the weight matrices.

Instead of saving k_1 floating-point numbers for $\mathbf{w}$, we now only need to save $2s$ elements: s elements are floating-point numbers representing the non-zero values in the sparse code, and the remaining s elements are integers that give the indices of those non-zero values. The indices can often be represented with 16-bit precision, unlike the non-zero values in the sparse code, which require 32-bit floating-point precision. At the RX end, $\mathbf{x}$ must be converted back to $\mathbf{w}$. This requires the sensing matrix $\mathbf{A}$, which is random and can be controlled by a seed to reproduce the exact $\mathbf{w}$ using $\mathbf{w} = \mathbf{A}\mathbf{x}$. Thus, the receiver only needs $\mathbf{x}$ to obtain $\mathbf{w}$. This process can be viewed as a method of uncovering the inherent sparsity within natural signals, as represented through the weight space of INRs. As we hypothesized, the

ability to condense natural signals into a dictionary hinges on identifying specific patterns encoded within the weights of INRs. Once the non-zero elements of the sparse vector are obtained at the RX, the resulting procedure is virtually the same across different INR-based baselines. Our method fundamentally achieves compression by delving into the weight spaces to uncover patterns, a step not typically taken by existing methods. A summarization of **SINR** is illustrated in Fig. 2. As can be seen from Fig. 2, **SINR** is only dependent on the weights of the INR and is applied prior to any quantization or entropy coding schemes. Therefore, **SINR** can be applied to existing INR compression methods to improve their compressibility.

3.3. How much fundamental compression does **SINR** achieve?

3.3.1. Standard INRs

Consider an INR with l hidden layers, yielding $l + 2$ total layers. For simplicity, assume k neurons per hidden layer. If the input dimension is a and the output dimension is b , the total number of weight parameters is given by $\mathcal{T}_s = a \times k + l \times k^2 + b \times k$. However, **SINR** modifies this structure by reducing the parameters from $\mathcal{T}_s$ in the original network to $\mathcal{T}_{s_{\text{SINR}}} = a \times 2s + k \times l \times 2s + b \times 2s$, where $s \ll k$. Additionally, **SINR** does not require transmitting any additional data to recover the original INR weights.

3.3.2. Tiny INRs

Let us define an INR as “tiny” if the number of neurons in a hidden layer, denoted by k , is less than 50. In such cases, we aim to achieve a sparse representation where $2s < k$ and $\|x\|_0 = s$. However, achieving a sparse representation that satisfies $2s < k$ is often extremely challenging and typically does not result in effective compression. To overcome this, we exploit the fact that the weight matrix connecting the i^{th} layer to the $(i + 1)^{\text{th}}$ layer is of dimensions $k \times k$. By vectorizing this weight matrix, we obtain a vector of dimension $k^2 \times 1$. Given that k^2 is significantly larger than k , we can apply our **SINR** procedure directly to the flattened weight matrix. This strategy leads to a sparser representation, thus enhancing compression efficiency for tiny INRs.

3.3.3. COIN++

In the COIN++ framework, modulation parameters are stored instead of traditional weights and biases, under the assumption that the base network parameters can be transmitted beforehand. For n test images, each segmented into m patches with a latent dimension of size d , COIN++ necessitates the transmission of $m \times d$ parameters for reconstructing each image. As the base network in COIN++ conforms to a standard INR structure, it is amenable to further compression via the **SINR** technique. By implementing **SINR** principles on the modulations in COIN++, the parameter transmission requirement per image can be reduced from

$m \times d$ to just $2s \times d$, where $s \ll m$. As the size of each test image and the number of images in the test dataset grow, COIN++ would typically require the transmission of numerous parameters. However, by leveraging **SINR**, both the modulations and the base network can be significantly compressed, achieving enhanced compression.

3.4. Quantization and entropy coding

After an INR is trained, its parameters are not immediately saved but are first subject to quantization [12]. This involves reducing the bitwidths below typical floating-point precision. Following quantization, the parameters are processed through entropy coding, in our experiments we utilize Brotli coding [1, 18], which allows the compressed data to be stored or transmitted efficiently. To retrieve the original parameters, the decoder must reverse the entropy coding and then perform dequantization. In the case of **SINR**, the compression process is intensified by utilizing the sparsity induced in the model parameters by natural signals. Once the sparse code is established, the parameters are quantized and subjected to entropy coding. The decoder then reverses the entropy coding and dequantizes the data. Finally, the model parameters are reconstructed by multiplying them with a random Gaussian matrix with a specific seed.

4. Experiments

4.1. Experimental setup

SINR, a novel INR compression algorithm, is predicated on the idea that if natural signals are compressible through a dictionary, then INRs should be similarly compressible. This concept underpins **SINR**’s goal to efficiently reduce INR storage requirements while maintaining high fidelity. Our experiments, conducted using the PyTorch framework following WIRE [29] codebase on an NVIDIA RTX A6000 GPU, spanned various data types including images, occupancy fields, and neural radiance fields. Image encoding metrics involved file size, bits per pixel (bpp) and Peak Signal-to-Noise Ratio (PSNR). Occupancy fields were evaluated using file size and Intersection over Union (IoU), and neural radiance fields were assessed using file size and PSNR. Other than the network configurations mentioned in the paper, for occupancy field evaluation, we utilized an MLP with 128 hidden neurons, and 3 hidden layers. For INRIC, we applied the network hyperparameters specified in its paper. In COIN++, we followed the guidelines in its paper but modified the hidden neuron size to 300. All experiments used Brotli entropy coding with a 16-bitwidth (65536 levels) uniform quantizer.

4.2. How do we find s ?

We implemented L_1 minimization using the Orthogonal Matching Pursuit (OMP) algorithm [38]. The OMP algorithm requires the pre-determination of s before obtaining

$\mathbf{x}$, and it must adhere to the condition $2s < k_1$. If $2s$ is set too low, it results in inaccurate representations of $\mathbf{w}$ within the weight space. Therefore, we incrementally increased s from a low value until $2s = k_1$ for all KODAK images in the C_1 experiment, as outlined in Sec. 4.3. Our findings suggest that the optimal value of s for successfully reconstructing the weight space does not depend on the specific image but on the number of neurons in a hidden layer. By adjusting the neuron count, we identified an optimal s that accurately reconstructs the weight space while satisfying the specified constraint. Extending these experiments to natural signals outside the KODAK dataset confirmed the consistency of our results. Additionally, we have included a regression plot in the supplementary that details how to determine the optimal s based on the number of neurons.

4.3. Image encoding

Representing an image through the weights and biases of a neural network serves as a method of encoding. For our image encoding task, we utilized the KODAK dataset, which includes 24 natural RGB images, each measuring 768×512 pixels. We conducted five types of experiments, denoted as C_i , where i ranges from 1 to 5, to demonstrate the effectiveness of our proposed method.

Experiment C_1 involved encoding each image in the KODAK dataset using an INR without positional embedding, by varying the number of neurons in each hidden layer. Experiment C_2 mirrored C_1 , but with the variation in the number of hidden layers instead. Experiments C_3 and C_4 implemented the meta-learning approach for INRs proposed in INRIC, without and with positional embedding for the input layer, respectively. Experiment C_5 involved the COIN++ framework, testing both with and without patching. When using patching, we adopted 32×32 patches as suggested by COIN++. However, we observed that without patching, even as the latent modulation dimension increased, the average PSNR obtained by COIN++ remained nearly constant. For these meta-learning-based experiments, we used the first 12 images of the KODAK dataset for meta-learning and the remaining 12 images for fine-tuning. For all image encoding experiments, we used the Sinusoidal activation function (see supplementary).

Let us define h and m as the number of hidden layers and the number of neurons per hidden layer in an INR, respectively. For experiment C_1 , we configured the INR with settings (h, m) as $(2, 32)$, $(3, 64)$, $(3, 128)$. Experiment C_1 aims to assess the effectiveness of **SINR** by varying the number of hidden neurons. The results, depicted in Fig. 3, demonstrate how effectively **SINR** identifies the compressibility of the weight space. This is indicated by the bpp values, which reflect the size of the model parameters. For example, representing the KODAK dataset with an average PSNR of 30 dB requires about 3.7 bpp for COIN and 2.0

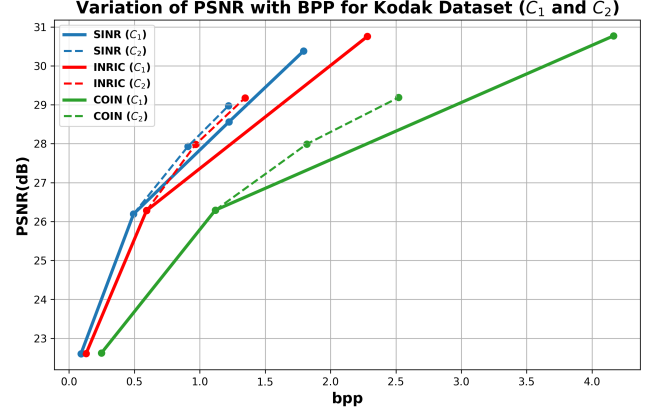


Figure 3. **Experiments C_1 and C_2 : Identifying compressible INR combinations.** The **SINR** approach demonstrates that configurations in C_1 are more compressible than those in C_2 . Furthermore, in both configurations **SINR** achieves lower bpp while maintaining the PSNR values.

bpp for INRIC. However, **SINR** significantly reduces the bpp to approximately 1.7 using the same quantizer and entropy coder. The first configuration in C_1 falls under the category of tiny INRs, underscoring the proposed method’s effectiveness even for compact INRs. As illustrated in Fig. 3, **SINR** achieves the same level of PSNR as baselines with a lower bpp for any network configuration. This substantial reduction of bpp across the C_1 experiment showcases the efficiency and compactness achieved by **SINR**. From C_1 , it can be established that greater compressibility of an INR into a dictionary is possible with an increased number of hidden neurons. Following the conclusions drawn from experiment C_1 , experiment C_2 was designed to explore the impact of increasing the number of hidden layers on the effectiveness of **SINR**. The configurations tested in C_2 were $(h, m) = \{(3, 64), (5, 64), (7, 64)\}$. As illustrated in Fig. 3, **SINR** consistently achieved PSNR levels comparable to baseline methods, but with a reduced bpp. Given that C_2 maintained a constant neuron count at 64, the observed deviations in compression between **SINR** and INRIC were less significant than those observed in C_1 . This discrepancy can be attributed to the following: an INR configuration with a higher number of neurons (e.g., $m = 128$), even with fewer hidden layers (e.g., $h = 2$), possesses more trainable parameters. Consequently, such a model is capable of learning a more robust representation of the image compared to configurations with a larger number of layers but fewer neurons per layer. As a result, the compressible characteristics of the images are more effectively transferred into the model parameters during the INR training process. This leads to a more compressible INR. These findings support the premise that if natural images can be efficiently compressed into a dictionary, the weight space

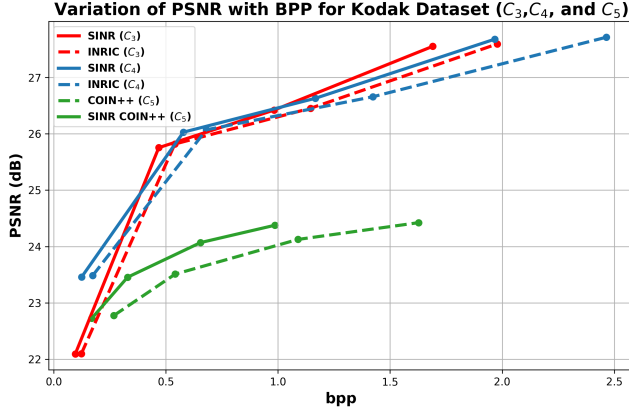


Figure 4. **Experiments C_3 , C_4 , and C_5 : Identifying compressible INR combinations under Meta-Learning.** Meta-learning approaches have been introduced for INRs to enhance their generalization abilities and achieve faster convergence. When assessing induced sparsity in the weight space, **SINR** demonstrates a significant reduction in bpp values while maintaining nearly the same PSNR performance as the baselines.

of INRs can also be effectively compressed.

As in previous experiments, each image required separate training of an INR. Experiments C_3 and C_4 address this challenge through meta-learning, with and without positional embedding, respectively. The configurations for these INRs are given by $(h, m) = \{(3, 32), (3, 64), (3, 96), (3, 128)\}$. For COIN++, the number of layers was set to 5 with MLP’s hidden dimension at 300. The latent dimension parameter (d) varied as follows: $d = \{16, 32, 64, 96\}$. Fig. 4 presents the experimental results for C_3 , C_4 , and C_5 , illustrating significant compression capabilities of the proposed **SINR** within a meta-learning framework. Notably, models using positional embedding generally have more parameters than those without.

Comparing the performance of INRIC and **SINR** without positional embedding schemes, the initial INR configuration shows that **SINR** exhibits a lower bpp for the same average PSNR. Generally, as bpp increases, the representation capacity of the INR enhances, leading to more robust image representation. From the graphs, as bpp increases, **SINR** demonstrates greater improvements in PSNR than INRIC, a phenomenon that can be explained by the aforementioned logic. In the case of COIN++, the approach focuses on fine-tuning only the modulations using their proposed meta-learning method. However, since fine-tuning encodes natural signals within these modulations, they should also be compressible via a dictionary. Due to patching, each KODAK test image results in a $d \times 384$ matrix. Our experiments reveal that these modulations encode hidden redundancies in natural signals. For instance, to achieve an average PSNR of approximately 24.2 dB,

COIN++ requires more than 1.5 bpp; however, the same PSNR can be achieved with COIN++ using just under 1 bpp by exploiting the hidden sparsity in its modulations through our proposed approach. Therefore, when a high-capacity model effectively represents a signal, it must encapsulate this sparsity within its weight and bias spaces. **SINR** explores and removes redundancies in these parameters, retaining only essential information. Fig. 5 showcases the decoded images by **SINR** alongside with the INR based image compressors. Decoded PSNR, BPP, and file size are displayed in the first, second, and third rows of the text boxes.

4.4. Occupancy fields encoding

Occupancy fields are represented by binary values, either 1 or 0, where 1 denotes that the signal lies within a specified region and 0 indicates its absence. Another variant of occupancy volumes stores not only the presence or absence of a signal but also the color at that location. Typically, occupancy fields consume more space than other data modalities. However, they can be represented with higher accuracy and lower storage requirements using INRs. In this experiment, we followed the sampling procedure described in [29]. Occupancy fields can be thought of as representations of three-dimensional objects, capturing natural signals. Despite following the sampling procedure, redundancies may exist that are not essential for representing the occupancy volume. Identifying these redundancies can reduce storage requirements. However, identifying them in the spatial domain (xyz) requires domain-specific algorithms, as described in Sec. 1. As INRs serve as unified data modality representors, these redundancies must be encapsulated within its weights space. **SINR** fundamentally compresses the INRs into a dictionary regardless of the data modality; therefore, indeed it is equally applicable to occupancy fields. To validate this hypothesis for occupancy fields, two experiments were conducted using shapes from the Stanford shape dataset [33]. Figure 6 showcases the decoded **SINR**’s representations for ‘Thai Statue’ (first volume) and ‘Lucy’ (fourth volume) datasets alongside the existing INR-based occupancy compressor. We use the Gaussian activation function for this task (see supplementary). The first value and second value in each text box represent the IoU metric and storage requirement, respectively, except for GT.

4.5. Additional materials

The pseudocode for **SINR**, additional results and ablation studies on finding s are in the supplementary material.

5. Conclusion

Implicit Neural Representations (INRs) have emerged as a promising framework for unified data modality representation. Several studies have explored the potential for compressing images, occupancy fields, and audio using INRs.

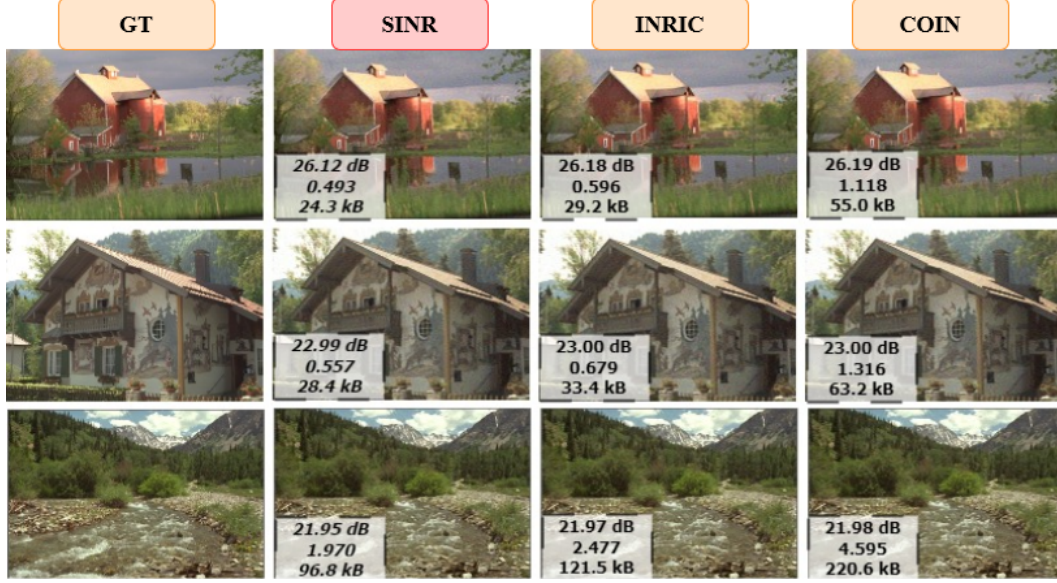


Figure 5. **Results for image encoding experiment.** **SINR** compresses the INR into a dictionary, significantly reducing the storage required compared to baseline INR image compressors. The results demonstrate that the decoded representations undergo a very negligible loss in PSNR, which is minimal considering the substantial storage space saved.

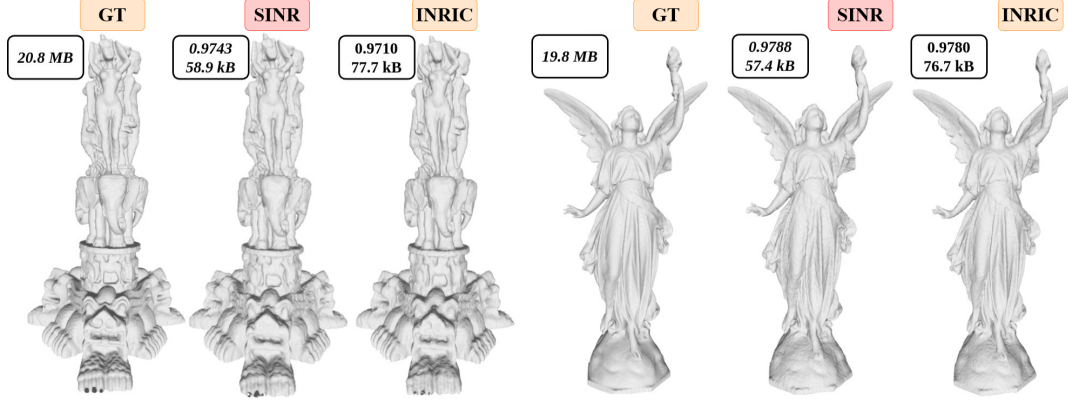


Figure 6. **Results for occupancy fields encoding experiment.** The results clearly demonstrate that **SINR** achieves the smallest file size and the highest accuracy metric for every shape in the tested dataset. The significant compression obtained by our algorithm suggests that occupancy fields, when represented using an INR, can be more efficiently compressed into a dictionary compared to images.

However, none of these methods have investigated whether the INR itself can be compressed prior to quantization and entropy coding. As natural signals can be efficiently compressed in bases of transformed domains due to their sparsity—allowing for higher accuracy and lower storage requirements—we hypothesize that a similar compressible nature must also exist in the INR once it is trained. With the discovery that weight vectors in the weight space tend to adhere to a Gaussian distribution, we propose **SINR**, which compresses any INR in a dictionary. Furthermore, we demonstrate that this dictionary does not need to be learned but can instead be generated using a seed. We compare our findings with standard INR compressors for im-

ages, occupancy fields, and neural radiance fields. **SINR** achieves fundamental compression for any INR, independent of other post-processing methods such as quantization and entropy coding, and it showcases significantly lower storage requirements and higher fidelity across various data modalities. Through our experiments, we observed that the INR can be more compressed when a more robust representation of the signal is learned. Additionally, some data modalities exhibit greater compressibility than others. We firmly believe this research will aid other researchers in exploring more patterns in the weight spaces of INRs and in developing operators and transforms for INR.

Acknowledgment

This work is supported by the Intelligence Advanced Research Projects Activity (IARPA) via Department of Interior/ Interior Business Center (DOI/IBC) contract number 140D0423C0076. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright annotation thereon. Disclaimer: The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of IARPA, DOI/IBC, or the U.S. Government.

References

- [1] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. Brotli: A general-purpose data compressor. *ACM Transactions on Information Systems (TOIS)*, 37(1):1–30, 2018. 5
- [2] David Alexandre, Chih-Peng Chang, Wen-Hsiao Peng, and Hsueh-Ming Hang. An autoencoder-based learned image compressor: Description of challenge proposal by nctu. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 2539–2542, 2018. 1
- [3] Selen Ayas and Murat Ekinici. Single image super resolution using dictionary learning and sparse coding with multi-scale and multi-directional gabor feature representation. *Information Sciences*, 512:1264–1278, 2020. 3
- [4] Karlheinz Brandenburg. Mp3 and aac explained. In *Audio Engineering Society Conference: 17th International Conference: High-Quality Audio Coding*. Audio Engineering Society, 1999. 1, 2
- [5] Komaravolu Chandrasekharan. *Classical Fourier Transforms*. Springer Science & Business Media, 2012. 4
- [6] Zhengxue Cheng, Heming Sun, Masaru Takeuchi, and Jiro Katto. Energy compaction-based image compression using convolutional autoencoder. *IEEE Transactions on Multimedia*, 22(4):860–873, 2019. 1
- [7] Zhengxue Cheng, Heming Sun, Masaru Takeuchi, and Jiro Katto. Learned image compression with discretized gaussian mixture likelihoods and attention modules. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7939–7948, 2020. 1
- [8] Fabio Duarte. Amount of data created daily (2024). <https://explodingtopics.com/blog/data-generated-per-day>, 2024. Accessed: 2024-07-14. 1
- [9] Emilien Dupont, Adam Goliński, Milad Alizadeh, Yee Whye Teh, and Arnaud Doucet. Coin: Compression with implicit neural representations. *arXiv preprint arXiv:2103.03123*, 2021. 2
- [10] Emilien Dupont, Hrshikesh Loya, Milad Alizadeh, Adam Goliński, Yee Whye Teh, and Arnaud Doucet. Coin++: Neural compression across modalities. *arXiv preprint arXiv:2201.12904*, 2022. 2
- [11] Sharath Girish, Abhinav Shrivastava, and Kamal Gupta. Shacira: Scalable hash-grid compression for implicit neural representations. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17513–17524, 2023. 2
- [12] Robert M. Gray and David L. Neuhoff. Quantization. *IEEE transactions on information theory*, 44(6):2325–2383, 1998. 5
- [13] Zekun Hao, Arun Mallya, Serge Belongie, and Ming-Yu Liu. Implicit neural representations with levels-of-experts. *Advances in Neural Information Processing Systems*, 35:2564–2576, 2022. 2
- [14] Daniel J Hsu, Sham M Kakade, John Langford, and Tong Zhang. Multi-label prediction via compressed sensing. *Advances in neural information processing systems*, 22, 2009. 3
- [15] Junlin Hu and Yap-Peng Tan. Nonlinear dictionary learning with application to image classification. *Pattern Recognition*, 75:282–291, 2018. 3
- [16] Zhe Hu, Jia-Bin Huang, and Ming-Hsuan Yang. Single image deblurring with adaptive dictionary learning. In *2010 IEEE International Conference on Image Processing*, pages 1169–1172. IEEE, 2010. 3
- [17] Donghua Jiang, Nestor Tsafack, Wadii Boulila, Jawad Ahmad, and JJ Barba-Franco. Asb-cs: Adaptive sparse basis compressive sensing model and its application to medical image encryption. *Expert Systems with Applications*, 236: 121378, 2024. 3
- [18] Gareth A Jones and J Mary Jones. *Information and coding theory*. Springer Science & Business Media, 2012. 5
- [19] Ashish Kapoor, Raajay Viswanathan, and Prateek Jain. Multilabel classification using bayesian compressed sensing. *Advances in neural information processing systems*, 25, 2012. 3
- [20] Syed Ali Khayam. The discrete cosine transform (dct): theory and application. *Michigan State University*, 114(1):31, 2003. 4
- [21] Li Liu and Paul Fieguth. Texture classification using compressed sensing. In *2010 Canadian Conference on Computer and Robot Vision*, pages 71–78. IEEE, 2010. 3
- [22] Yang Liu, Qingchao Chen, Wei Chen, and Ian Wassell. Dictionary learning inspired deep network for scene recognition. In *Proceedings of the AAAI conference on artificial intelligence*, 2018. 3
- [23] Liyan Ma, Lionel Moisan, Jian Yu, and Tiejong Zeng. A dictionary learning approach for poisson image deblurring. *IEEE Transactions on medical imaging*, 32(7):1277–1289, 2013. 3
- [24] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 2
- [25] David Minnen, Johannes Ballé, and George D Toderici. Joint autoregressive and hierarchical priors for learned image compression. *Advances in neural information processing systems*, 31, 2018. 1

- [26] AKM Rabby and Chengcui Zhang. Beyondpixels: A comprehensive review of the evolution of neural radiance fields. *arXiv preprint arXiv:2306.03000*, 2023. 2
- [27] AM Raid, WM Khedr, Mohamed A El-Dosuky, and Wesam Ahmed. Jpeg image compression using discrete cosine transform-a survey. *arXiv preprint arXiv:1405.6147*, 2014. 2
- [28] Sameera Ramasinghe and Simon Lucey. Beyond periodicity: Towards a unifying framework for activations in coordinate-mlps. In *European Conference on Computer Vision*, pages 142–158. Springer, 2022. 2, 3
- [29] Vishwanath Saragadam, Daniel LeJeune, Jasper Tan, Guha Balakrishnan, Ashok Veeraraghavan, and Richard G Baraniuk. Wire: Wavelet implicit neural representations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18507–18516, 2023. 2, 3, 5, 7
- [30] K Seemakurthy, A Majumdar, J Gubbi, NK Sandeep, A Varghese, S Deshpande, M Girish Chandra, and P Balamurali. Deep dictionary learning for inpainting. In *Computer Vision, Pattern Recognition, Image Processing, and Graphics: 7th National Conference, NCVPRIPG 2019, Hubballi, India, December 22–24, 2019, Revised Selected Papers 7*, pages 79–88. Springer, 2020. 3
- [31] Aviv Shamsian, Aviv Navon, David W Zhang, Yan Zhang, Ethan Fetaya, Gal Chechik, and Haggai Maron. Improved generalization of weight space networks via augmentations. *arXiv preprint arXiv:2402.04081*, 2024. 2
- [32] Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in neural information processing systems*, 33:7462–7473, 2020. 2, 3, 1
- [33] Stanford University Computer Graphics Laboratory. The stanford 3d scanning repository. <https://graphics.stanford.edu/data/3Dscanrep/>. Accessed: 2024-07-15. 7
- [34] Yannick Strömpler, Janis Postels, Ren Yang, Luc Van Gool, and Federico Tombari. Implicit neural representations for image compression. In *European Conference on Computer Vision*, pages 74–91. Springer, 2022. 2
- [35] Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems*, 33:7537–7547, 2020. 2, 3
- [36] Hao Tang, Hong Liu, Wei Xiao, and Nicu Sebe. When dictionary learning meets deep learning: Deep dictionary learning and coding network for image recognition with limited data. *IEEE transactions on neural networks and learning systems*, 32(5):2129–2141, 2020. 3
- [37] Lucas Theis, Wenzhe Shi, Andrew Cunningham, and Ferenc Huszár. Lossy image compression with compressive autoencoders. In *International conference on learning representations*, 2022. 1
- [38] Joel A Tropp and Anna C Gilbert. Signal recovery from random measurements via orthogonal matching pursuit. *IEEE Transactions on information theory*, 53(12):4655–4666, 2007. 5
- [39] Gregory K Wallace. The jpeg still picture compression standard. *IEEE transactions on consumer electronics*, 38(1): xviii–xxxiv, 1992. 1
- [40] Jan Hendrik Weissenbruch. The shipping canal at rijswijk, known as ”the view at geestbrug”. Painting. 2
- [41] Yueqi Xie, Ka Leong Cheng, and Qifeng Chen. Enhanced invertible encoding for learned image compression. In *Proceedings of the 29th ACM international conference on multimedia*, pages 162–170, 2021. 1
- [42] Xijuan Zhang, Oscar L Olvera Astivia, Edward Kroc, and Bruno D Zumbo. How to think clearly about the central limit theorem. *Psychological Methods*, 2022. 2
- [43] Hongyi Zheng, Hongwei Yong, and Lei Zhang. Deep convolutional dictionary learning for image denoising. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 630–641, 2021. 3
- [44] Jinjia Zhou and Jian Yang. Compressive sensing in image/video compression: Sampling, coding, reconstruction, and codec optimization. *Information*, 15(2):75, 2024. 3
- [45] Fang Zhu, Shuai Guo, Li Song, Ke Xu, Jiayu Hu, et al. Deep review and analysis of recent nerfs. *APSIPA Transactions on Signal and Information Processing*, 12(1), 2023. 2